

Requirements Specification for a Graph Database Supporting X4Good Social Media Platform

1. Introduction

This document specifies the requirements for a graph database designed to support a next-generation social media platform, called X4Good. The platform is envisioned as a highly connected, AI-driven ecosystem where users interact through social relationships, communities, content sharing, immersive experiences, and intelligent recommendations.

The graph database serves as the core component responsible for modeling, storing, and querying complex relationships among users, content, devices, communities, and digital assets.

The proposed graph database must support:

- Billions of interconnected entities
- Real-time social interactions
- Personalized recommendations
- Community discovery
- Privacy-aware querying
- AI-driven analytics
- Distributed deployment

The system must provide low-latency graph traversals while maintaining scalability, reliability, and security.

2. Graph Data Model Requirements

2.1 Node Types

The graph database shall support the following node types:

Node Type	Description
User	Platform user
Post	Published content
Comment	User comments
Community	User groups
Topic	Subject category
Hashtag	Content tag
Event	Online or virtual event

Node Type	Description
Device	User device
Location	Geographic location
Media	Images, videos, and multimedia
Advertisement	Sponsored content

Additional node types may be added without schema redesign.

2.2 Relationship Types

The graph database shall support directed relationships including:

Relationship	Description
FOLLOWS	User follows another user
FRIEND_OF	Mutual friendship
LIKES	User likes content
SHARES	User shares content
COMMENTS_ON	User comments on content
POSTED	User creates content
MEMBER_OF	Community membership
TAGGED_IN	User tagged in content
BLOCKED	User blocks another user
MUTED	User mutes another user
VIEWED	User viewed content
RECOMMENDED	Recommendation relationship
SIMILAR_TO	Similarity relationship

Relationships should present temporal and contextual properties.

Example:

```
(:User)-[:LIKES {
  timestamp: datetime(),
  reaction: "love"
}]->(:Post)
```

3. Functional Requirements

Functional requirements should be implemented in cypher.

3.1 User Management

The system shall support:

- User registration
- User profile management
- Relationship management

3.2 Content Management

The system shall support:

- Content creation
- Content sharing
- Content tagging

The database shall support:

- Likes
- Comments
- Shares
- Follows

3.3 Recommendation Engine Support

The system shall support queries such as:

- Recommended friends
- Recommended communities
- Recommended content

Example of friend suggestion:

```
MATCH (u:User)-[:FOLLOWS]->(:User)-[:LIKED]->(p:Post)
WHERE NOT (u)-[:LIKED]->(p)
RETURN p
```

4. Example of generic cypher commands

```
(:User)-[:FOLLOWS]->(:User)
(:User)-[:POSTED]->(:Post)
(:User)-[:LIKED]->(:Post)
(:User)-[:MEMBER_OF]->(:Community)
(:Post)-[:HAS_TOPIC]->(:Topic)
(:User)-[:USES_DEVICE]->(:Device)
(:User)-[:LOCATED_IN]->(:Location)
```